

Az eldönthetőség determinisztikus és nemdeterminisztikus formái, eldönthetetlen problémák. Algoritmusok idő és tárigényének definíciója. A P és az NP bonyolultsági osztályok. Hatékony visszavezetés, teljesség, NP-teljes problémák.

AZ ELDÖNTHETŐSÉG DETERMINISZTIKUS ÉS NEMDETERMINISZTIKUS FORMÁI, ELDÖNTHETETLEN PROBLÉMÁK

Ahhoz, hogy matematikailag egzakt módon beszélhessünk algoritmusokról, szükségünk van egy modellre. A Church-Turing tézis kimondja, hogy minden, ami intuitív értelemben algoritmussal kiszámítható, az kiszámítható Turing-géppel is.

Turing-Gép

- a Turing-gép egy absztrakt automata, amely egy végtelen szalagon dolgozik: $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$, ahol:
 - Q : az állapotok véges halmaza
 - Σ : a bemeneti ábécé (nem tartalmazza az üres jelet és a start jelet)
 - Γ : a szalagábécé (tartalmazza Σ -t és az üres jelet valamint a start jelet is)
 - δ : az átmeneti függvény: $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, _, R\}$
 - ha a számítás egy pillanatában a gép a q állapotban van és az olvasófej alatti cellában az a szimbólum áll, és $\delta(q, a) = (p, b, d)$, akkor a gép átmegy a p állapotba, az a szimbólumot ebben a cellában átírja b -re (persze lehet $a = b$ is), és a d irányba lépteti a fejet (egyet balra, jobbra vagy helyben marad)
 - nemdeterminisztikus esetben: $Q \times \Gamma \rightarrow P\{Q \times \Gamma \times \{L, _, R\}\}$, mivel több irányba mehet
 - q_0, q_{acc}, q_{rej} : kezdő-, elfogadó- és elutasító állapotok
- lehetőség bement: 
- eldönthetőség vs felismerhetőség
 - **eldönthető** (rekurzív)
 - ha létezik olyan Turing-gép, amely a nyelv minden szavára ($w \in L$) véges lépésben megáll és elfogad, minden nem a nyelvbe tartozó szóra pedig megáll és elutasít
 - **felismerhető** (rekurzívan felsorolható)
 - ha a nyelvbe tartozó szavakon megáll és elfogad, de a nyelvben nem létező szavakon pedig vagy elutasít, vagy végtelen ciklusba kerül

Eldönthetőség fogalma

- egy probléma (vagy nyelv) akkor eldönthető, ha létezik hozzá olyan algoritmus (vagy Turing gép), amely minden lehetséges bemenetre:
 1. Véges időn belül leáll (nincs végtelen ciklus)
 2. Minden outputja a helyes választ adja meg (igen vagy nem, azaz elfogadja vagy sem)

Eldönthetlenség

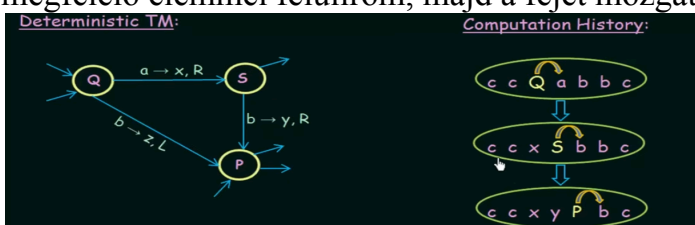
- akkor eldönthetetlen egy probléma, hogy ha van is eldöntő algoritmus nem biztos, hogy minden output-ja helyes, vagy lehet, hogy végtelen ciklusba eshet
 - ez gyakran az **önreferencia** miatt alakul ki: a probléma "saját magába harap", és egy logikai ellentmondáshoz (paradoxonhoz) vezet, ha megpróbáljuk megoldani
- legismertebb ilyen eldönthetetlen probléma a **megállási probléma** (Halting)
 - $L_h = \{ \langle M, w \rangle \mid M \text{ megáll a } w \text{ bemeneten} \}$
 - Alan Turing bizonyította be, hogy nem létezik olyan program, amely képes lenne tetszőleges programról és annak bemenetéről megmondani, hogy az valaha leáll-e, vagy végtelen ciklusba kerül-e
 - adott egy M program (Turing-gép kódja) és egy x bemenet. Eldönthető-e, hogy M megáll-e (elfogadja vagy elutasítja-e és végtelen ciklusba nem eshet) a x bemeneten?
 - válasz: Nem
 - indirekt módon, a diagonalizációs módszerrel bizonyítjuk
 - tegyük fel, hogy létezik egy M_0 gép, amely eldönti a problémát
 - ekkor csinálhatunk egy D ("Diagonális") gépet, amely M programot kapja inputba és az M_0 programot, gépet futatja amely M -et futatja és ekkor:
 - ha M_0 szerint megállna, D végtelen ciklusba lép;
 - ha M_0 szerint nem állna meg, D megáll

$D(M) = \text{if } M_0(M; M) \text{ then } \nearrow \text{ else halt.}$

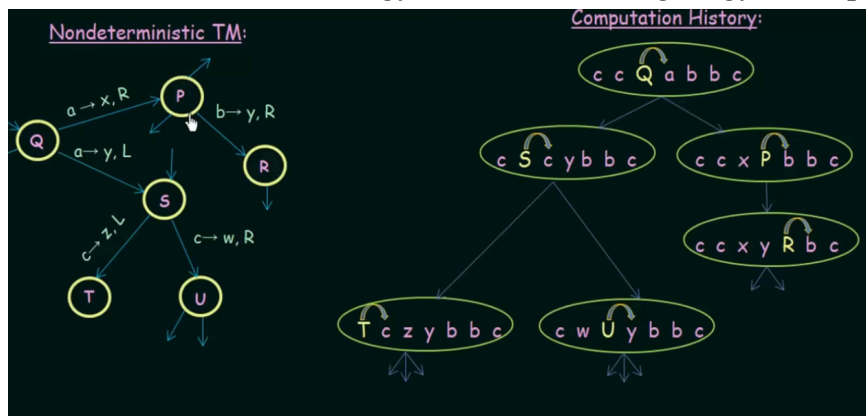
- ellentmondás lép fel ezért M nem létezhet
- Rice tétele kimondja, hogy a Turing gépek nyelveinek minden nem-triviális tulajdonsága eldönthetetlen
 - a triviális problémák: az egyik az, melynek minden lehetséges input „igen” példánya, a másik pedig az, melynek minden lehetséges input „nem” példánya. (tehát az egyik az, melyet eldönt a return true; kód, a másik pedig, melyet eldönt a return false;)
 - az összes többi nem-triviális és automatikusan igaz, hogy semmi nemtriviálisat nem tudunk eldönteni egy program viselkedéséről a forráskódja ismeretében

Determinizmus

- a **determinisztikus algoritmusok** olyan algoritmusok melyek során minden pillanatban pontosan tudjuk, mi a következő lépés
 - egy adott állapotból egy adott bemeneti jelre **csak egyetlen út** vezet a következő állapotba
 - azonos bementre mindig ugyanazt a választ adja
- a számítási előzményekben (computation history) látszik az aktuális állapot és az, hogy mi a következő input, ezt az inputot a determinisztikus automata élében lévő megfelelő elemmel felülírom, majd a fejet mozgatom a megfelelő irányba



- a **nemdeterminisztikus algoritmusok** olyan algoritmusok melyek futás során nem egyetlen döntéssorozatot követnek, hanem különböző lehetséges döntések közül választhatnak és mindegyiket párhuzamosan vizsgálhatják
 - csak akkor dönthető el nemdeterminisztikus algoritmussal, ha determinisztikussal is eldönthető
 - nemdeterminisztikus algoritmus időigény szerint sokszor hatékonyabb
 - egy adott állapotból egy bemeneti jelre több irányba is mehetünk egyszerre, vagy akár tippelhetünk is
 - az algoritmus akkor fogad el egy bemenetet, ha létezik *legalább egy olyan út*, ami elfogadó állapotba vezet
 - mint egy fa ahol minden ágat egyszerre próbálunk ki



- **Hamilton út**
 - menete
 - a bemeneti gráfra generálunk egy N elemű tömböt, aminek minden elemére generálunk egy $1 + \lceil \log(N) \rceil$ bites számot (ezzel 1 és N közötti számot kapunk). Ezt követően mondjuk egy 5 csúcsú gráfra kaphatunk egy ilyen tömböt: [1, 5, 2, 4, 3] (nem bitek)
 - generálás után validáció következik (bemeneti gráf csúcsainak egy permutációját (sorbarendezését) kaptuk-e)
 - Ellenőrizzük, hogy a két egymást követő csúcsok szomszédosak-e és az út minden csúcsot érint-e: ha igen, akkor találtunk egy Hamilton-utat, adjunk vissza TRUE-t, ha nem, akkor ez épp nem volt jó, és ez a szál visszaad egy FALSE-t, és értékelődik ki a többi
 - minden lépés egyenként polinom idejű, szóval az egész polinom idő alatt eldönthető
 - nemdeterminisztikusan polinomidőben eldönthető problémák az NP osztályba tartoznak
- **3-színezés**
 - adott egy G gráf, kiszínezhetőek-e a csúcsai három színnel úgy, hogy két szomszédos csúcs ne legyen azonos színű
 - menete
 - N elemű tömb létrehozása, mindegyikhez egy kétbites számot rendelünk
 - ellenőrizni kell, hogy a gráf minden (i, j) élére $T[i] \neq T[j]$, ha mindig igaz, akkor TRUE, különben FALSE
 - lineáris idő alatt lefut

ALGORITMUSOK IDŐ- ÉS TÁRIGÉNYE

- az algoritmusok idő- és tárigénye fontos tényező, ezek határozzák meg, hogy egy algoritmus milyen gyorsan és mennyi memóriát felhasználva oldja meg a problémát
 - a teljesítmény optimalizálása és az algoritmusok hatékonyságának szempontjából fontosak: pl. mennyire skálázható nagy bemeneti méreteknél

Időigény ($\text{TIME}(f(n))$)

- az algoritmus futási idejének mértékét a bemenet (input) függvényébe adjuk meg (algától eltérően)
- az időkomplexitást gyakran nagy O jelöléssel (Big O) fejezik ki, mely a legrosszabb esetbeli futási időt adja meg
 - input méret az, hogy hány biten tudjuk leírni az összes bejövő számot, jelölése n ennek függvényében adjuk meg az időigényt
 - pontos felső korlát helyett nagyságrend
- fontos, ha van benne ciklus, ami végtelen ciklusba eshet valamilyen input hatására (ha csak egy is van ilyen) akkor az algoritmus nem korlátos időigényű, nincs korlátja, nincs időigény
- Időigény = hányszor fut le
- Futásidő = n szerinti futásidő, az n az input méret, akár $n-1$ vagy $n-2$ is lehet

Időigény Számítása:

- Ciklusoknál összeszorozzuk a ciklusmag időigényét azzal ahányszor lefut
- Szekvenciálisan (egymás utáni kódblokkok) az időigényeket összeadjuk
- T int tömb esetén
 - $T.\text{length}$ -re egy jó becslés mindig az $O(n)$, azaz lineáris
 - $T[i]$ -re, azaz egy tömb elem értékére pedig az $O(2^n)$

Időigény futásidővé alakítása:

- ciklusnak meg van hányszor fut le, ez az időigénye (pl: $O(\log A)$ vagy $O(A)$, stb.), ebbe kell belehelyettesíteni azt, hogy az input hány bites
($n = O(\log A)$, átalakítva $A = 2^n$)
- lehet 1 de akár 2 vagy több input is, a lényeg, hogy a legrosszabb esetet kell vizsgálni

```

1 int isPrime(int A) {
2     for (int B := 2; B < A; B++) {
3         if (A % B == 0) return 0
4     }
5     return 1
6 }

```

```

1 int topBit(int A) {
2     int B := 1
3     while (B + B <= A) {
4         B := B + B
5     }
6     return B
7 }

```

Elemzése:

- isPrime
 - minden konstans idő, kivétel a ciklus az $B = 2$ -től indul és mindig egyet adunk hozzá és A -ig fut ezért $O(A - 2)$ alkalommal fut, ami $O(A)$
 - az A input bitbe kifejezve $n = O(\log A)$, amiből jön, hogy $A = O(2^n)$
 - ezt visszahelyettesítjük abba, hogy hány alkalommal fut le és azt kapjuk $O(A)$ -ból, hogy $O(2^n)$ alkalommal fut le így a ciklusmaggal kell már csak összeszoroznom, mely konstans szóval $O(2^n) * O(1) = O(2^n)$ időigényű
- topBit
 - minden konstans idő, kivétel a ciklus az $B = 1$ -től indul és mindig duplázuk és addig fut még a B duplája kisebb vagy egyenlő mint A ezért $O(\log A)$ alkalommal fut le
 - az A input bitbe kifejezve $n = O(\log A)$, amiből jön, hogy $A = O(2^n)$
 - ezt visszahelyettesítjük és azt kapjuk $O(\log A)$ -ból, hogy $O(\log 2^n) = O(n)$ alkalommal fut le így a ciklusmaggal kell már csak összeszoroznom, mely konstans szóval $O(n) * O(1) = O(n)$ időigényű

3. feladat.

Elemizzük az alábbi kód időigényét!

```

1 int mod(int A, int B) {
2     while (B <= A) {
3         A := A - B
4     }
5     return A
6 }

```

Elemzése:

- 3.feladat
 - nincs időigénye, mivel $B = 0$ esetén végtelen ciklusba kerülünk
- 4. feladat
 - minden konstans idő kivétel a ciklus, mely $O(A/B)$ alkalommal futhat le
 - legrosszabb eset itt ha az A nagy, a B pedig kicsit
 - az A inputot $n = O(\log A)$ biten tárolhatjuk, tovább alakítva $A = O(2^n)$
 - a B input legrosszabb esetben $B = 3$, mivel előtte lévő számok if-fel kizárva, ezért $B = 3$
 - az így kapott értékeket csak behelyettesítjük, azaz $O(A/B)$ az időigény akkor $O((2^{n-2} - 1) / 3) = O(2^n)$

4. feladat.

Elemizzük az alábbi kód időigényét!

```

1 int mod(int A, int B) {
2     if (B <= 2) return 0
3     while (B <= A) {
4         A := A - B
5     }
6     return A
7 }

```

Elemzése:

```
int tombIncSum(int[] T) {  
    int sum := 0  
    for (int i := 0; i < T.length; i++) {  
        while (T[i] > 0) {  
            sum := sum + 1  
            T[i] := T[i] - 1  
        }  
    }  
    return sum  
}
```

- minden konstans idejű, ám van 2 egymásba ágyazott ciklus
 - külső for ciklus a T tömb hosszáig megy, eggyel lépegetve, ez azt jelenti, hogy $T.length$ becsléséig ami $O(n)$ a futásiidőigénye
 - belső while addig megy még a T tömb egy eleme nagyobb mint 0, a lépés köz -1, azaz $T[i]$ becslését használhatjuk, ami $O(2^n)$
- ezeket mivel egymásba vannak össze kell szorozni + a maradék $O(1)$ konstans időigények: $O(1) + (O(n) * O(2^n))$, ami $O(n * 2^n)$, tehát időigénye ennek az algoritmusnak ennyi

Hierarchia példákkal:

- Konstans idő ($O(1)$):
 - Egy lista első elemének lekérdezése mindig ugyanannyi időt vesz igénybe, függetlenül a lista méretétől. Nincs szükség az egész lista bejárására
- Logaritmikus idő ($O(\log n)$):
 - Bináris keresés egy rendezett listában úgy működik, hogy minden lépésben megfelel a keresési tartományt. Ezáltal a keresési idő arányosan csökken a lista méretének logaritmusával
- Lineáris idő ($O(n)$):
 - Egy lista összes elemének végigellenőrzése azt jelenti, hogy minden elemet egyesével meg kell vizsgálni. Ezért az időráfordítás egyenesen arányos a lista hosszával
- Lineáris-logaritmikus idő ($O(n \log n)$):
 - A quicksort algoritmusnál minden elosztási lépés során a lista elemeit két alcsoportra osztjuk, majd rekurzívan rendezzük őket. Az elosztási lépések száma logaritmikus, és minden lépés lineáris időt vesz igénybe.
- Négyzetes idő ($O(n^2)$):
 - A buborékrendezés során minden elemhez többszörösen végig kell menni a listán, összehasonlítva és szükség szerint cserélve az elemeket. Ez két egymásba ágyazott ciklust eredményez, mindkettő $O(n)$ iterációval
- Polinomiális idő ($O(n^k)$):
 - A gráf szélességi keresése (BFS) egy gráf minden csúcsát és élét legfeljebb egyszer látogatja meg. Az időráfordítás így a csúcsok és élek számával arányosan növekszik, ami polinomiális időt eredményez
- Exponenciális idő ($O(2^n)$):
 - Az utazó ügynök problémánál minden lehetséges útvonalat ki kell próbálni. Mivel az összes lehetséges útvonalak száma exponenciálisan nő a városok számával, az időráfordítás is exponenciális
- Faktoriális idő ($O(n!)$):
 - Teljes permutációkat vizsgáló algoritmusoknál (pl a TSP teljes keresése) az összes lehetséges sorrendet meg kell vizsgálni. A permutációk száma faktoriálisan növekszik az input mérettel, ami faktoriális időt eredményez

Tárigény ($\text{SPACE}(f(n))$)

- az algoritmus által igényelt memória mennyisége a bemenet méretének függvényében
- a tárkomplexitás szintén nagy O jelöléssel van kifejezve
 - pl az $O(n)$ azt jelenti, hogy a memóriaigény lineárisan nő a bemenet méretével
- input méret az, hogy hány biten tudjuk leírni az összes bejövő számot, jelölése n .

Tárigény Számítása:

- amennyiben az inputot csak olvassuk akkor az outputot és az inputot se kell beleszámolni
 - ha írjuk is akkor még az input változó tárigénye is hozzáadódik
- egy változó tárigénye:
 - pl: i változó az input A változóig megy akkor maximális értéke bitben kifejezve $n = O(\log A)$, ami $A = O(2^n)$ (bit)
 - a bitet tárigényé kell alakítani (logaritmizálni), tehát ha $O(2^n)$ bit volt akkor $O(\log 2^n) = O(n)$ lesz a példa szerinti változó tárigénye
- algoritmus tárigénye:
 - ha nincs benne függvényhívás akkor a lokális változók összege
 - ha vannak benne függvényhívások akkor azon függvényeknek ki kell számítani a tárigényét külön-külön és azon értékek közül a legnagyobbat hozzá kell adni az eredeti kódunk tárigényéhez
 - a belső függvények tárigénye = a változók tárigénye + az argumentumok tárigénye
 - rekurzió esetén pedig a függvény tárigénye = (változók tárigénye + argumentumok tárigénye) * rekurzió maximális mélysége

Hierarchia példákkal:

- Konstans tár ($O(1)$):
 - Lista első elemének lekérdezése: Egy lista első elemének lekérdezése esetén nincs szükség további memórafoglalásra, az algoritmus egy előre meghatározott kis mennyiségű memóriát használ
- Logaritmikus tár ($O(\log n)$):
 - Bináris keresés egy rendezett listában: A bináris keresés egy rendezett listában rekurzív megközelítéssel csak a logaritmus szerinti lépéseket igényli a keresési intervallum felosztásához, és az egyes rekurzív hívások egy kis fix memóriaterületet igényelnek
- Lineáris tár ($O(n)$):
 - Lista összes elemének másolása: Ha egy lista összes elemét egy új listába kell másolni, akkor az új lista ugyanannyi elemet fog tartalmazni, mint az eredeti, tehát a tárigény lineáris lesz
- Lineáris-logaritmikus tár ($O(n \log n)$):
 - Mergesort algoritmus: A mergesort algoritmus logaritmikus rekurzív hívásokat és lineáris segédtárolókat használ az egyes alrészek összeillesztéséhez, így a tárigénye lineáris-logaritmikus

- Négyzetes tár ($O(n^2)$):
 - Mátrix szorzás: Két $n \times n$ méretű mátrix szorzása esetén az eredmény egy $n \times n$ méretű mátrix lesz, ami n^2 memóriaterületet igényel az eredmények tárolásához
- Polinomiális tár ($O(n^k)$):
 - Gráfok ábrázolása szomszédsági mátrixszal: Egy gráf összes lehetséges csúcsának és élének tárolása egy $n \times n$ méretű mátrixban polinomiális tárigényű, ahol n a csúcsok száma
- Exponenciális tár ($O(2^n)$):
 - A hatványhalmaz előállítása: Az összes részhalmaz generálása egy n elemű halmazból 2^n részhalmazzal eredményez, mindegyiket külön tárolva exponenciális tárigényt eredményez
- Faktoriális tár ($O(n!)$):
 - Teljes permutációk tárolása: Egy n elemű halmaz összes permutációjának tárolása $n!$ különböző sorrend tárolását jelenti, ami faktoriális tárigényt eredményez

NEVEZETES BONYOLULTSÁGI OSZTÁLYOK: P, NP

Eldönteni vs megoldani?

- **eldönteni (NP osztály)**
 - döntési problémák esetén az algoritmusnak IGEN-t vagy NEM-et kell válaszol adnia egy kérdésre (accept vagy reject, tehát bináris)
 - az "eldönteni" tehát azt jelenti, hogy meghatározzuk, hogy létezik-e egy megoldás, amely megfelel a feltételeknek
- **megoldani (P osztály)**
 - optimalizálási problémák esetén az algoritmusnak egy konkrét megoldást kell találnia (nem csak eldöntenie, hogy létezik-e)
 - például az utazóügynök probléma optimalizálási változata az, hogy találjuk meg a legrövidebb lehetséges útvonalat, amely érinti az összes várost pontosan egyszer

P osztály (Polinomiális idő)

- könnyen kezelhető problémák
 - de van ami nem gyors pl: $O(n^{100})$ futás idejű algoritmus P-beli, de gyakorlatban használhatatlan, de ritka az ilyen
- azok a problémák tartoznak ide, melyekre létezik determinisztikus algoritmus, ami a problémát **meg tudja oldani** polinomidőben
 - **összeadás**, két szám összeadása lineáris ($O(n)$) időbe elvégezhető
 - **keresés rendezett listában**, pl bináris keresés rendezett listában $O(\log n)$ időben találja meg az elemet, n a lista mérete
 - **legnagyobb közös osztó (GCD)**, az euklideszi algoritmus $O(\log \min(a, b))$ időben számítja ki két szám legnagyobb közös osztóját
 - **prim szám-e**

- miért polinom?
 - polinomok zártak az egymásba ágyazásra
 - ha egy algoritmus hív egy másikat az eredmény még mindig polinom
 - különböző számítási modellek közötti átváltás is legfeljebb polinom idejű változást okoz
- P és az NP bonyolultsági osztályok közel állnak egymáshoz (egyesek szerint azonosak, de nincs bizonyítás még erre)
 - P-ben a polinomidőben megoldható problémák vannak
 - NP-ben a polinomidőben leellenőrizhető (eldönthető) problémák vannak
 - valószínűleg $P \neq NP$, azaz léteznek olyan problémák, amelyek megoldását könnyű ellenőrizni, de nehéz megtalálni
 - ellenőrzés könnyebb, mint megoldásokat keresni
- $P \subseteq NP$ (részhalmaz), mivel ha valamit megoldok polinomidő alatt, akkor le is ellenőrzöm az alatt
- **Cobham-Edmonds tézis**
 - P-ben van az összes olyan eldöntési probléma, amire létezik polinom időigényű ($O(n^k)$) eldöntő algoritmus valamilyen k konstans-ra
 - kezelető problémák tartoznak a P-be

NP osztály (Nemdeterminisztikus polinomiális idő)

- könnyen ellenőrizhető (eldönthető) problémák
- azok a problémák tartoznak ide, amelyekre létezik nemdeterminisztikus algoritmus, ami a problémát polinomidőben **el tudja dönteni**
 - más, hogy magyarázva: Tanú / Verifikáció
 - nemdeterminisztikusan generálunk egy tanút, megoldást egy IGEN példány, fontos, hogy polinomiális időben kell maradnia, azaz nem lehet túl hosszú
 - létezik olyan konstans, hogy (x, y) esetén, ahol x az input és y a tanú akkor $|y| \leq |x|^k$
 - majd determinisztikusan, polinomiális időben ellenőrizni tudjuk, hogy a tanú valóban helyes (polinomiális időben verifikáló algoritmus)
 - csak akkor van NP-be egy probléma, ha létezik polinomiális idejű verifikáló algoritmus és egy “tanú” (megoldás), ezt a tanút polinomiális időben kell tudni ellenőrizni

Idő vs tár: idő és tár között az alapvető különbség, hogy a tár újrafelhasználható

SPACE osztály (Lineáris tár)

- azon problémákat tartalmazó bonyolultsági osztály, amelyekre létezik egy $O(f(n))$ lineáris tárigényű program
- NP és SPACE között nem ismert összefüggés, annyi biztos, hogy ezek nem egyenlőek, illetve ezen felül NP zárt a visszavezetésre, míg a SPACE nem

PSPACE osztály (Polinomiális tár)

- polinomiális tárban megoldható problémák tartoznak bele
- $NP \subseteq PSPACE$, (NP részhalmaza $PSPACE$ -nek)
- Sakk
 - meghatározni, hogy adott állásból a fehér (vagy fekete) nyerhet-e biztosan
 - bár a pontos futási idő exponenciális lehet, polinomiális tárban megoldható, tehát $PSPACE$ -teljes

NPSPACE osztály (Nemdeterminisztikus polinomiális tár)

- Nemdeterminisztikus algoritmussal, polinomiális tárban megoldható problémák
- Savitch tétele kimondja, hogy a $PSPACE = NPSPACE$
 - időbonyolultság esetén ez úgy van ($P \neq NP$ -vel)

Összefoglaló hierarchia és egyéb osztályok

- hierarchia:
 - $L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE(= NPSPACE) \subseteq EXP$
- **R, RE**: Eldönthető és felismerhető problémák
- **TIME(f(n)), NTIME(f(n))**: Determinisztikus/nemdeterminisztikus időbeli bonyolultsági osztályok
- **SPACE(f(n)), NSPACE(f(n))**: Determinisztikus/nemdeterminisztikus tárbeli bonyolultsági osztályok
- **P = TIME(n^k)**: Polinomiális időben eldönthető problémák
- **NP = NTIME(n^k)**: Nemdeterminisztikus polinomiális időben eldönthető problémák
- **L = SPACE(log(n))**: Logaritmikus tárban eldönthető problémák
- **NL = NSPACE(log(n))**: Nemdeterminisztikus logaritmikus tárban eldönthető problémák
- **PSPACE = SPACE(n^k)** – Polinom tárban eldönthetőek
- **NPSPACE = NSPACE(n^k)** – Nemdeterminisztikus polinom tárban
- **EXP = TIME($(2^n)^k$)**: Exponenciális idő alatt eldönthető problémák

HATÉKONY VISSZAVEZETÉS, TELJESÉG ÉS NP-TELJES PROBLÉMÁK

Visszavezetés (redukció)

- egy probléma másik problémára történő átalakítását jelenti, jelölés: $A \leq B$
 - A visszavezethető B -re
 - $x \in A \Leftrightarrow f(x) \in B$
 - vesszük az A probléma egy tetszőleges x bemenetét
 - átalakítjuk az f transzformációs függvényrel egy $f(x)$ bemenetű ami B bemenete lesz
 - feltétel, hogy kiszámítható legyen f
 - odaadjuk $f(x)$ -et a B problémát megoldó algoritmusnak
 - ha B elfogadja, akkor A is elfogadja; ha elutasítja, A is elutasítja
 - fordítva is, azaz igazságtartó
- ha van egy B problémánk, amiről már tudjuk, hogy eldönthetetlen (pl. a megállási probléma), és egy A problémánk, amiről be akarjuk látni, hogy eldönthetetlen
- megmutatjuk, ha meg tudnánk oldani B -t, akkor meg tudnánk oldani A -t is ($A \leq B$)
 - mivel B -t nem tudjuk megoldani, így A -t sem lehet
- olyan eszköz amivel megmutathatjuk, hogy egy feladat legalább olyan nehéz mint egy másik, azaz ha A visszavezethető B -re akkor B legalább olyan nehéz mint A
 - ha B -t gyorsan megtudjuk oldani akkor A -t is

Hatékony visszavezetés

- az algoritmusok hatékony visszavezetése, egy olyan visszavezetés mely hatékony, ami annyit jelent, hogy polinomiális időben, azaz $O(n^k)$ végrehajtható
 - gyakran használt módszer a bonyolultsági osztályok közötti kapcsolatok vizsgálatában, például az NP-teljes problémák esetén
- definíció szerint az A eldöntési probléma hatékonyan (=polinomidőben) visszavezethető a B eldöntési problémára ($A \leq_p B$), ha van olyan transzformációs függvény ami:
 - polinomidőben kiszámítható
 - ha A -ra nincs polinomidejű algoritmus akkor B -re sincs
 - A inputjaiból B inputjait készíti választartó módon
- a hatékony visszavezetés **transzitiv**
 - ahhoz, hogy megmutassuk, hogy A probléma C nehézségű, vissza kell vezetnünk A -ra egy biztosan C -nehéz problémát, ez a B lesz
 - mivel B -re minden C -beli visszavezethető, ezáltal ha B is visszavezethető A -ra akkor minden C -beli is visszavezethető A -ra

- **zárttság visszavezetésre**
 - ha $A \leq_p B$ és $B \in C$, akkor $A \in C$, azaz ugyanabba az osztályba vannak, más szóval ha egy probléma benne van akkor a nála gyengébbek is
 - ha egy C osztály zárt a visszavezetésre és A egy C -teljes probléma akkor C -ben lévő minden probléma visszavezethető A -ra (A probléma reprezentálja az osztályt)
 - P, NP, R és RE is zárt
- példák
 - **Maze $\leq_p$ Sokoban**
 - a cél mellé két új mező, egyikbe golyó, másikba lyuk
 - **Hamilton-út $\leq_p$ TSP(E)**
 - városok súlya megkapható módosított szomszédsági gráf segítségével, 0: önmaga, 1: volt a két csúc között él, 2: egyébként
 - TSP(E) második paramétere $N + 1$
 - **Párosítás $\leq_p$ SAT**
 - élekre felveszünk egy változót, igazra állítjuk, ha a párosításban benne volt, különben hamis
 - ezt követően leellenőrizzük, hogy a csúcsokra pontosan egy él illeszkedik

Definíciók: a C egy probléma osztály, az A egy eldöntendő probléma

- egy A probléma akkor **C-nehéz**, ha C minden problémája visszavezethető rá A -ra
 - $A' \leq_p A$
- egy A probléma akkor **C-teljes**, ha C -nehéz, és ő maga (A) is benne van C -ben
 - tehát például az NP-teljes problémák, olyan problémák, amelyre minden NP-ben lévő probléma polinomiális időben visszavezethető (C -nehéz), és önmaga (A) is NP-ben vannak
 - mikor hasznos ha valami teljes?
 - **problémák megértése**
 - ha egy probléma teljes egy osztályban (pl NP-teljes), akkor a probléma megértése segíthet abban, hogy jobban megértsük az egész osztály jellemzőit és nehézségeit
 - **visszavezetés**
 - ha egy probléma teljes egy osztályban, akkor ugyanolyan bonyolultsági osztályba tartozó problémákat is vissza tudunk vezetni rá
 - megkönnyítheti a problémák közötti kapcsolatokat és átfogó megoldások megértését
 - **algoritmusfejlesztés**
 - teljes problémák tanulmányozása és megoldása új algoritmusok és megközelítések kidolgozását ösztönözheti, amelyeket más hasonló problémákra is alkalmazhatunk

NP-teljes probléma

- egy L nyelv NP-teljes, ha:
 - $L \in \text{NP}$ (benne van az osztályban), azaz polinomidőben ellenőrizhető, ÉS
 - L NP-nehéz
 - minden $L' \in \text{NP}$ nyelvre teljesül, hogy $L' \leq_p L$, azaz minden más NP-beli probléma visszavezethető rá
 - van olyan NP-nehéz probléma ami nem NP-teljes (pl: megállási probléma)
 - megállási probléma azért nem jó mert ugyan minden NP-beli probléma visszavezethető rá (NP-nehéz), de nem NP-teljes mert nincs benne NP osztályba (sőt, eldönthetetlen)
- azok a problémák tartoznak ide, amelyek a legnehezebbek az NP osztályba
 - ha bármelyikre találnánk egy polinomidejű algoritmust, azzal bebizonyítanánk, hogy $P = \text{NP}$
 - nem minden NP-beli feladat NP-teljes (pl P-beli problémák is benne vannak NP-be, de azok nem NP-teljesek, mert könnyebbek)

Nevezetes NP-teljes problémák

- 3-SAT (3-kielégíthetőség) $(x_1 \vee x_2 \vee \bar{x}_3) \wedge (x_2 \vee x_3 \vee \bar{x}_4) \wedge (x_1 \vee \bar{x}_2 \vee x_4)$
 - Adott egy logikai formula (konjunktív normálformában)
 - 3-SAT egy speciális esete a SAT-nak, egy 3-literal CNF formula van
 - Létezik-e a változóknak olyan igaz/hamis értékadása, amely igazra értékeli az egész formulát (formula kielégíthető-e)
- Utazó ügynök probléma (Travelling Salesman Problem, TSP)
 - egy adott városokat és az utak költségeit tartalmazó gráfban keressük a legkisebb összköltségű kört
 - eldöntési verziója NP
 - létezik-e olyan útvonal, amely nem haladja meg az adott K hosszúságot, tehát itt nem a legkisebbet keressük
 - speciális esete a Hamilton-kör probléma
- Hamilton-kör probléma (akár Hamilton-út)
 - egy gráfban eldönteni, hogy létezik-e olyan kör amely minden csúcsot pontosan egyszer érint
- Klikk probléma
 - adott egy gráf és egy k szám, létezik-e a gráfban k csúcsból álló teljes részgráf (ahol mindenki mindenkivel össze van kötve)
- Gráf színezés
 - adott egy gráf és egy k szám, kiszínezhető-e a gráf k színnel úgy, hogy semelyik két szomszédos csúcsnak ne legyen azonos a színe

NP-teljes visszavezetések

- **Általános megoldáskeresése**

- ha egy új problémát NP-teljesre vezetünk vissza, akkor bármilyen NP-teljes problémára már meglévő megoldási technikát alkalmazhatunk
- Pl: Utazó ügynök probléma (Traveling Salesman Problem, TSP)
 - NP-teljes utazó ügynök probléma során egy ügynöknek kell meglátogatnia egy sor várost úgy, hogy minden várost pontosan egyszer érint és a teljes utazási költség minimális legyen
 - ha új hasonló probléma merül fel, pl drónnak kell csomagokat kézbesítenie különböző helyekre, akkor ez a probléma visszavezethető a TSP-re, így a TSP-re kidolgozott heurisztikákat és közelítő algoritmusokat lehet alkalmazni

- **Optimalizálás és heurisztikák**

- pl: raktár optimalizálás
 - az NP-teljes probléma, mint a készletelhelyezési probléma (knapsack problem), segíthet a raktárak optimalizálásában, ahol egy meghatározott kapacitású raktárba a legnagyobb értékű árukat kell rakni
 - Heurisztikus és közelítő algoritmusok alkalmazásával, mint a DP (dinamikus programozás) vagy a genetikus algoritmusok, hatékonyabb megoldásokat találhatunk, amelyek közel optimálisak

- **Kutatási irányok**

- segíthet új kutatási irányokat kijelölni, például új bonyolultsági osztályok felfedezésében vagy új algoritmikus technikák kidolgozásában
- pl: számítógépes biztonság
 - sok számítógépes biztonsági probléma, mint például a szubgráf-izomorfizmus probléma (subgraph isomorphism problem), NP-teljes. Ez a probléma azt vizsgálja, hogy egy adott gráf tartalmaz-e egy másik gráffal izomorf részgráfot
 - az ilyen problémák tanulmányozása új titkosítási módszereket és biztonsági protokollokat inspirálhat, amelyek nehezebbé teszik a támadók számára az algoritmusok kijátszását